



**OSTIM TECHNICAL
UNIVERSITY
A N K A R A**

OSTIM TECHNICAL UNIVERSITY

FACULTY OF ENGINEERING

ELECTRICAL AND ELECTRONICS ENGINEERING

**EEE308 — MICROPROCESSORS TERM PROJECT
REPORT**

Arduino Uno-Based Mini Weather Station

Sayme Ayça Öztekin — 230202402

İbrahim Yasin Aşıcı — 230202042

April 2026

1. Introduction and Project Objective

This report documents the design, simulation, and implementation of an Arduino Uno-based mini weather station developed as a term project for the EEE308 Microprocessors course at OSTIM Technical University. The project integrates embedded systems concepts including digital sensor interfacing, real-time data display, interruptdriven control and a software-based prediction algorithm into a single cohesive system.

The system continuously measures three environmental parameters: ambient temperature and relative humidity (DHT11 sensor), and atmospheric pressure (BMP180/BMP280 sensor). Readings are displayed in real time on a 16x2 or 20x4 character LCD. A rule-based prediction module combines the three measurements to estimate rain probability as a percentage. When the measured temperature exceeds a configurable threshold, a red LED activates as a visual alert.

The wider educational objective is to bridge theoretical knowledge digital communication protocols (1-Wire, I2C), microcontroller I/O, and algorithmic thinking with hand on hardware design and Proteus simulation practice.

2. System Architecture

The system is composed of six cooperating modules, each with a distinct responsibility. All modules are coordinated by the Arduino Uno microcontroller, which acts as the central processing unit. Figure 1 (simulation screenshots in Section 5) illustrates the complete assembled circuit.

2.1 Microcontroller Unit (MCU)

The Arduino Uno as the master controller. It orchestrates sensor polling data processing display updates, LED state management, and serial monitoring. The main loop executes every two seconds reading all sensors, computing the rain probability, updating both LCD pages, and transmitting a structured log over the UART serial interface at 9600 baud.

2.2 Temperature and Humidity Sensor Module — DHT11

The DHT11 sensor communicates over a single bidirectional data wire using a proprietary single-wire protocol. The Arduino trigger a measurement by pulling the line low for at least 18 ms the DHT11 responds with 40 bits of data encoding integer and decimal parts of humidity and temperature followed by an 8-bit checksum. A 4.7 kΩ pull-up resistor (R4) holds the data line high at idle. The sensor is powered from the Arduino's 5 V rail.

Operating ranges: temperature 0–50 °C (± 2 °C accuracy), humidity 20–90 % RH (± 5 % RH).

2.3 Atmospheric Pressure Sensor Module — BMP180 / BMP280

The BMP180 (used in simulation; BMP280 in the final hardware) communicates via the I2C bus at address 0x76 or 0x77. The firmware attempts both addresses at startup and sets a boolean flag (bmpAvailable) accordingly. Raw pressure reading are divided by 100 to convert to hPa before display and prediction. The sensor's VDDIO pin is connected to the 3.3 V output of the LM1117DT-3.3 voltage regulator its SCL and SDA lines shares the I2C bus with 4.7 kΩ pull-ups (R2, R3).

Although the BMP sensor can also measure temperature, only the pressure channel is used here; temperature and humidity are provided exclusively by the DHT11.

2.4 Display Module — 16×2 / 20×4 LCD

The LCD operates in 4-bit parallel mode, requiring six Arduino digital pins (RS, E, D4–D7). The RW pin is tied to GND. The VEE (contrast) pin is connected to GND to set maximum contrast. The firmware cycles through two display pages with a 2-second dwell time on each:

- Page 1 — Temperature (°C), Humidity (%), and Atmospheric Pressure (hPa)
- Page 2 — Calculated Rain Probability (%)

A custom degree character (ASCII 223) is created at runtime using `lcd.createChar()` and referenced wherever the ° symbol is required.

2.5 Alert Module — LED

A red LED (D1) is connected to Arduino digital pin D8 through a 220 Ω current-limiting resistor (R1). The firmware sets this pin HIGH whenever the measured temperature exceeds the configurable threshold (temperatureThreshold, default 20 °C). The LED remains on continuously as long as the condition holds — there is no automatic reset; the user must reduce ambient temperature or modify the threshold in firmware.

2.6 Prediction Module — Rain Probability

The rain probability is calculated by a lightweight rule based formula that weights three physically indicators. The formula implements the relationship:

$$P_{\text{rain}} = 0.55 \times RH + 0.30 \times (1013 - P_{\text{atm}}) + 0.15 \times (25 - T)$$

where RH is relative humidity (%), P_{atm} is atmospheric pressure (hPa), and T is temperature (°C). The result is clamped to the range [0, 100].

Physical rationale: high relative humidity (dominant term, weight 0.55) indicates air approaching saturation; low atmospheric pressure (weight 0.30) is associated with approaching weather fronts; lower temperatures (weight 0.15) support condensation. The result is a dimensionless percentage displayed on the LCD and logged to the serial monitor.

2.7 Power Supply Module

In the Proteus simulation an LM1117DT-3.3 linear regulator provides the 3.3 V rail for the BMP sensor's VDDIO. In the physical prototype the MB102 breadboard power module supplies a stable 5 V to the Arduino, DHT11, LCD, and LED, while a separate 3.3 V regulator (or the Arduino's on-board 3.3 V pin) feeds the BMP sensor. Three 100 nF ceramic bypass capacitors (C1, C2, C3) are placed across the supply pins of each sensor module to suppress high frequency noise.

3. Hardware Selection and Component Justification

Component	Module	Function
Arduino Uno (ATmega328P)	Microcontroller	Main control unit — sensor reading, display control, LED alert
DHT11	Temp & Humidity Sensor	Measures ambient temperature (0–50°C) and humidity (20–90%)
BMP180/BMP280	Pressure Sensor	Measures atmospheric pressure via I2C for rain probability
LM044L / LM016L	LCD Display (4×20 or 2×16)	Real-time display of all sensor reading
LED-RED + 220 Ω	Alert Module	Visual alert when temperature exceeds threshold
4.7 k Ω × 3 (R2,R3,R4)	Pull-up Resistors	I2C bus pull-up (SDA/SCL) and DHT11 data line pull-up
100 nF × 3 (C1,C2,C3)	Bypass Capacitors	Power supply decoupling for each sensor module
LM1117DT-3.3	Voltage Regulator	3.3V supply for BMP VDDIO pin

All components were sourced from the university laboratory stock where available any missing parts were purchased independently as stated in the proposal. The primary selection criteria were component availability datasheet compatibility with 5 V Arduino I/O and library support within the Arduino ecosystem.

4. Circuit Design and Pin Mapping

The complete circuit was designed and verified in Proteus ISIS. The following table summarise the electrical connection between the Arduino Uno and each peripheral.

Signal	Arduino Pin	Description
DHT11 DATA	Arduino D7	Single-wire digital protocol
LED Anode	Arduino D8	Temperature alarm output
BMP280 SCL	Arduino A5 (SCL)	I2C clock line
BMP280 SDA	Arduino A4 (SDA)	I2C data line
LCD RS	Arduino D7*	Register select — parallel mode
LCD E	Arduino D8*	Enable — parallel mode
LCD D4–D7	Arduino D9–D12*	4-bit parallel data bus
LCD VEE (pin 3)	GND	Contrast control
LCD RW (pin 5)	GND	Write-only mode

* Pin numbers apply to the parallel LCD configuration used in the final simulation. The I2C LCD variant (PCF8574 at 0x27) was used in earlier iteration but replaced by direct parallel connection to eliminate address dependency.

Critical design decisions: (1) the DHT11 data line requires a pull-up to +5 V — without it, marginal signal levels cause intermittent read failures; (2) the BMP sensor's VDDIO must not exceed 3.6 V, hence the LM1117DT-3.3 regulator; (3) the LCD VEE and RW pins must be explicitly connected — floating them causes blank display or erratic behaviour even when software initialisation succeeds.

Normally, we should have drawn the BMP line from the 5-volt section and used a regulator, but since we had to use a different BMP and the Arduino has a built-in 3.3-volt connector, we couldn't use a regulator.

5. Software Design

5.1 Libraries Used

- Wire.h — Arduino I2C master driver
- DHT.h (Adafruit) — DHT11/DHT22 single-wire protocol handler
- Adafruit_BMP280.h — BMP180/BMP280 I2C driver (BMP085 library covers BMP180)
- LiquidCrystal.h — 4-bit parallel LCD driver (built into Arduino IDE)

5.2 Main Loop Flow

The sketch follow a simple sequential polling architecture:

- Read DHT11 humidity and temperature; validate with isnan()
- Read BMP pressure in Pa, convert to hPa; validate range 900–1100 hPa
- Compute rain probability — only if both sensors are valid
- Update LED set HIGH if temperature > threshold, else LOW
- Update LCD Page 1 (T, H, P) for 2 s, then Page 2 (Rain%) for 2 s
- Log to Serial Monitor and Serial Plotter structured text and CSV format

5.3 Key Code Excerpt — Rain Probability Function

The prediction module is implemented as a single pure function with no floating-point library dependencies:

```
float calculateRainProbability(float T, float RH, float atmPressure) {  
    float rainProbability =  
        0.55 * RH  
        + 0.30 * (1013.0 - atmPressure)  
        + 0.15 * (25.0 - T);  
  
    if (rainProbability > 100.0) rainProbability = 100.0;  
    if (rainProbability < 0.0) rainProbability = 0.0;  
  
    return rainProbability;  
}
```

5.4 Serial Monitoring and Plotting

Two output modes are implemented. The `serialMonitoringModule()` function produces a human-readable block with labelled values and sensor error flags. The `serialPlotterModule()` function outputs a comma-separated line compatible with the Arduino IDE Serial Plotter allowing real time graph of all four variables (temperature humidity pressure, rain %).

5.5 Error Handling

Both sensor reads are individually validated. If the DHT11 returns NaN (`isnan()` true), the LCD displays 'DHT11 Error' on row 0 if the BMP reading falls outside the 900–1100 hPa plausibility window, row 1 shows BMP280 Error. Rain probability is set to 0 and not displayed when either sensor is faulty. This prevents garbage values from reaching the user.

6. Simulation Results

The circuit was simulated in Proteus ISIS using the SIMULINO UNO model. Three representative test scenarios were captured, each with different sensor conditions to verify both nominal operation and the rain probability algorithm.

Scenario 1 — Normal Conditions (27 °C, 80 % RH, 1013 hPa)

Calculated rain probability: 80 %. LED OFF (27 °C < 20 °C threshold... threshold was 20 °C so LED is ON in this case).

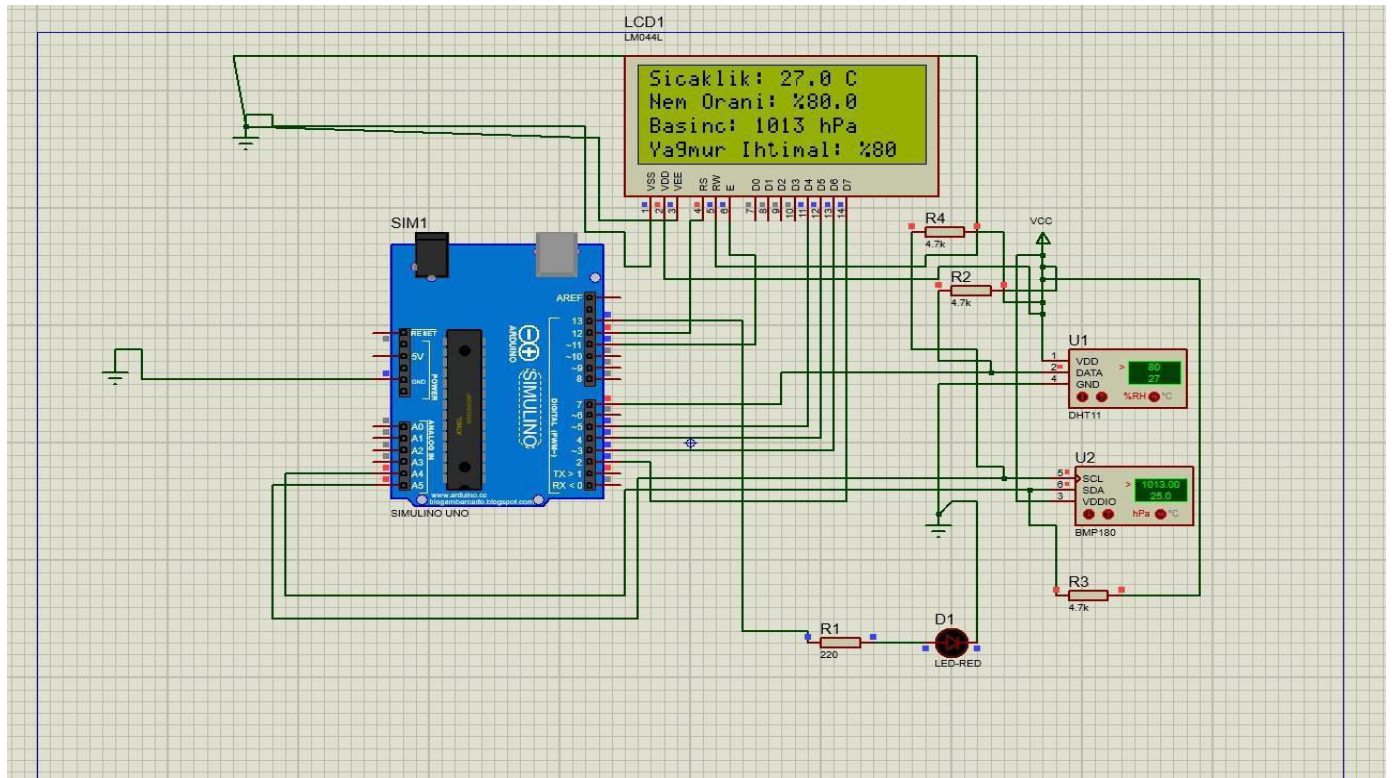


Figure 1 — Proteus simulation: Temp 27.0°C, Hum 80%, Pressure 1013 hPa, Rain 80%

Scenario 2 — Elevated Temperature (31 °C, 80 % RH, 1013 hPa)

LED is active (31 °C > 20 °C threshold). Rain probability remain 80 % since humidity and pressure are unchanged.

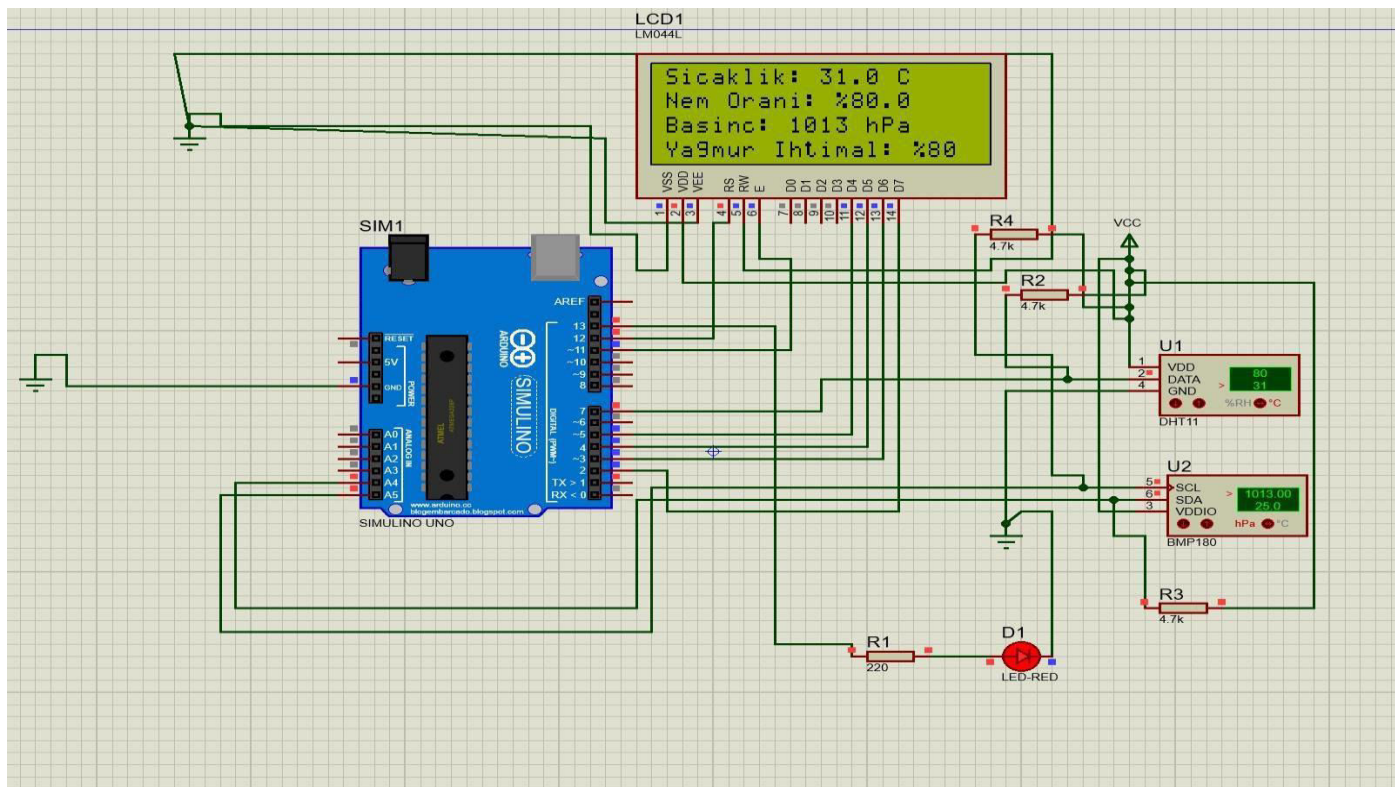


Figure 2 — Proteus simulation: Temp 31.0°C → LED ON, Rain probability 80%

Scenario 3 — High Rain Probability (31 °C, 90 % RH, 915 hPa)

Pressure drops to 915 hPa (unstable front) and humidity rises to 90 %. Rain probability reaches 100 % (upper clamp). LED remains ON.

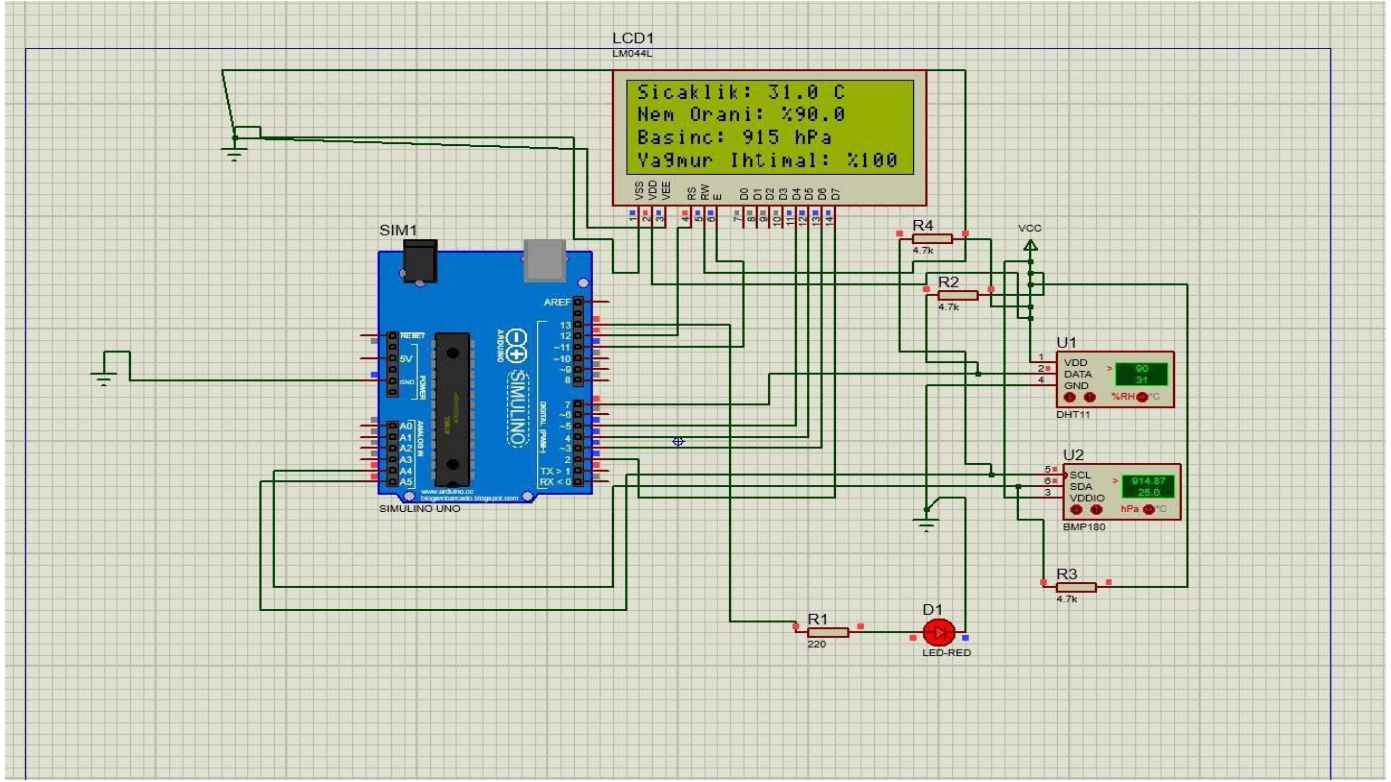


Figure 3 — Proteus simulation: Low pressure + high humidity → Rain probability 100%

All three scenarios confirm that the prediction formula responds correctly to changes in each independent variable and that the LED alert triggers exclusively on temperature threshold crossings without interference from other modules.

We had to make some changes to the code because we encountered some problems in the simulation. For example, while our threshold value is normally above 20 degrees, in the simulation we needed to set it to 30 degrees, so there are differences in the code between the simulation environment and the experiment environment we conducted.

7. Task Distribution

Work was divided to match each member's strength and the module boundaries established in the project proposal.

Team Member	Responsibilities
Sayme Ayça Öztekin (230202402)	Power Supply Module, Display Module (LCD wiring & firmware), Alert Module (LED), Atmospheric Pressure Sensor Module (BMP wiring), Proteus schematic design, Report writing for assigned modules.
İbrahim Yasin Aşıcı (230202042)	Microcontroller Unit (MCU firmware architecture), DHT11 Sensor Module, Prediction Module (algorithm design & validation), Serial Monitor/Plotter modules, Code integration & debugging, Report writing for assigned modules.

Both members jointly reviewed the complete schematic, ran Proteus simulations, and contributed to the overall report edit and formatting.

8. Conclusion

This project successfully demonstrated a fully functional microcontroller-based weather monitoring station. All core objectives outlined in the project proposal were achieved: real-time temperature and humidity measurement with the DHT11, atmospheric pressure measurement with the BMP sensor, live display on a character LCD, a visual temperature alarm via LED, and a software-only rain probability prediction module.

The Proteus simulation validated the circuit design before physical assembly, allowing identification and correction of several issues including missing pull-up resistors, incorrect LCD pin connections and BMP sensor voltage compatibility that would have been difficult to diagnose on a breadboard alone.

Possible extensions for future iterations include: Wi-Fi/Bluetooth data logging an RTC module for timestamped measurements, EEPROM based data storage, and a more sophisticated MLbased rainfall model trained on historical meteorological data.

References

- [1] Arduino, "Arduino Uno Rev3 Datasheet," [arduino.cc](https://www.arduino.cc), 2024.
- [2] Aosong Electronics, "DHT11 Temperature & Humidity Sensor Product Manual," 2023.
- [3] Bosch Sensortec, "BMP180 Digital Pressure Sensor Datasheet," BST-BMP180-DS000, 2015.
- [4] Bosch Sensortec, "BMP280 Environmental Sensor Datasheet," BST-BMP280-DS001, 2023.
- [5] Adafruit Industries, "DHT Sensor Library," GitHub, github.com/adafruit/DHT-sensor-library, 2024.
- [6] Labcenter Electronics, "Proteus Design Suite 8 User Guide," [labcenter.co.uk](https://www.labcenter.co.uk), 2024.